



UNIVERSIDADE FEDERAL DE SANTA CATARINA

Centro Tecnológico - Departamento de Informática e Estatística

Disciplina: INE5418 – Computação Distribuída

Data: 11 de maio de 2026

Trabalho 1: Encurtador de URLs Distribuído

Integrantes do grupo:

Fernanda Larissa Müller (21202109)

Julia Fischer Gazolla (23250586)

Lucas Tomio Schwchow (23250585)

1. Visão geral

Encurtador de URLs distribuído com três componentes que se comunicam por protocolos diferentes:

```
...  
[Clientes Python / JavaScript] --sockets TCP/JSON--> [Interceptador] --HTTP/REST--> [Servidor REST]  
...
```

- **Servidor REST (Python/Flask):** endpoints HTTP e armazenamento em dicionário em memória.
- **Interceptador (proxy) (Python):** recebe TCP dos clientes, aplica Cache-Aside (LRU) e Fila de Prioridades, repassa por HTTP ao servidor. O servidor não sabe que o proxy existe.
- **Bibliotecas cliente em Python e em JavaScript/Node.js:** mesma interface (``encurta``, ``resolve``, ``remove``) sobre o mesmo protocolo JSON/TCP.

Tudo roda em containers Docker orquestrados por ``docker-compose.yml``.

2. Decisões de implementação

2.1 Linguagens

- **Python 3.10** no servidor, no proxy e em uma das bibliotecas cliente. Familiaridade da equipe e disponibilidade direta de ``socket``, ``threading`` e ``queue`` na stdlib.
 - **Servidor:** Flask para os endpoints, por familiaridade da equipe; ``shortuuid`` para gerar códigos.
 - **Proxy:** ``requests`` para falar com o servidor; ``sockets`` e ``threading`` da stdlib.
- **Node.js 20** na segunda biblioteca cliente, sem dependências externas (só o módulo nativo ``net``).

2.2 Protocolo cliente ↔ proxy

JSON sobre TCP, uma requisição por conexão.

```
...  
// requisições  
{ "acao": "encurta", "url": "..." }  
{ "acao": "resolve", "codigo": "..." }
```

```

{"acao": "remove", "codigo": "..."}

// respostas
{"codigo": "...", "url_curta": "..."} // encurta
{"url_original": "..."} // resolve (miss)
{"url_original": "...", "fonte": "cache"} // resolve (hit)
{"removido": true} // remove
{"erro": "..."} // falha
...

```

O campo "fonte": "cache" é uma melhoria do projeto para observar o cache em ação sem precisar inspecionar logs do proxy.

2.3 Cache-Aside com política LRU (`proxy/cache\_aside.py`)

Classe `LRUCache` baseada em `collections.OrderedDict`. `get` faz `move\_to\_end` ao acessar; `put` insere no fim e remove o início (`popitem(last=False)`) ao exceder a capacidade; `invalidate` remove uma chave específica. Capacidade configurável via `CACHE\_CAPACITY` (padrão 5).

Sobre coerência: o proxy só invalida o cache depois que o `DELETE` no servidor retorna sucesso. Se a chamada HTTP falhar, a entrada antiga permanece no cache. Evita inconsistência transitória ("cliente acha que removeu mas o servidor ainda tem"), em troca de aceitar uma janela curta de dados desatualizados em caso de falha de rede.

2.4 Segundo padrão: Fila de Prioridades

Justificativa da escolha. Esse padrão faz sentido para uso no escopo desse projeto. O proxy recebe três tipos de operação com sensibilidades diferentes a latência:

- `resolve` (GET) — operação mais frequente e a que o usuário final percebe diretamente.
- `remove` (DELETE) — afeta coerência do cache; vale rodar antes dos `encurta`.
- `encurta` (POST) — tolera atraso. Ninguém colando uma URL pra encurtar nota 200ms a mais.

Implementação: uma `queue.PriorityQueue` armazena tuplas `(prioridade, ordem\_chegada, ...)`. Uma thread worker daemon consome a fila; a thread principal só aceita conexões e enfileira. Prioridades: `resolve=0`, `remove=1`, `encurta=2`.

Em uso normal a fila quase nunca acumula e o efeito é invisível. Para evidenciar o padrão durante a demo, criamos a variável de ambiente `DEMO\_DELAY` no proxy, que insere um atraso artificial em cada requisição (default 0). Detalhes na seção 4.4.

2.5 Configuração via variáveis de ambiente

Todos os parâmetros de runtime (portas, URL do servidor vista pelo proxy, capacidade do cache, `DEMO\_DELAY`) são lidos de variáveis de ambiente declaradas no `docker-compose.yml`, com defaults sensatos no código. Alterar um parâmetro não exige rebuild de imagem, apenas reiniciar o serviço com a nova env.

2.6 Identificação de códigos curtos

`shortuuid.uuid(name=url)[:8]` são 8 caracteres derivados de hash da URL. A mesma URL gera sempre o mesmo código, evitando duplicatas no `url\_storage`. Trocamos da escolha inicial (UUID v4 aleatório, 36 chars) porque códigos longos poluíam logs e exemplos.

3. Como compilar e executar

Instruções completas, com troubleshooting e descrição de cada componente, estão no `README.md` na raiz do projeto. Resumo:

```
```bash
docker compose up --build
```
```

Sobe os 4 serviços (server, proxy, client-python, client-js). Os dois clientes executam um exemplo automático e encerram; servidor e proxy ficam rodando.

Rodar um cliente isoladamente:

```
```bash
docker compose run --rm client-python
docker compose run --rm client-js
```
```

Bater direto na API REST (sem passar pelo proxy):

```
```bash
curl http://localhost:5050/urls
```
```

Encerrar:

```
```bash
docker compose down
```
```

macOS: a porta 5000 é ocupada pelo AirPlay Receiver, por isso o host mapeia 5050 → 5000 do container do servidor.

4. Exemplos de saídas de execução

Códigos curtos abaixo são valores reais retornados pelo sistema (8 caracteres `shortuuid`).

4.1 Cache HIT no `resolve`

```
```
$ docker compose run --rm client-python python -c "
from client import EncurtadorClient
c = EncurtadorClient()
r = c.encurta('https://example.com'); cod = r['codigo']
print('1a:', c.resolve(cod))
print('2a:', c.resolve(cod))
"
1a: {'url_original': 'https://example.com'}
2a: {'url_original': 'https://example.com', 'fonte': 'cache'}
```

...

A 2ª resolução vem com `"fonte": "cache"` e os logs do proxy mostram `[CACHE HIT]` sem novo `[CACHE MISS]` e não houve chamada HTTP ao servidor.

## 4.2 Invalidação no `remove` + eviction LRU (cliente Python completo)

O `\_\_main\_\_` do `clients/python/client.py` executa um ciclo completo: encurta + resolve para 6 URLs distintas, depois remove a última. Como `CACHE\_CAPACITY=5`, a 6ª inserção dispara eviction. Logs do proxy resumidos:

...

```
[CACHE MISS] ... 6eb12957 (inf)
[CACHE HIT] ... 6eb12957 (inf, move para MRU)
[CACHE MISS] ... 089be493 (ppgcc)
[CACHE MISS] ... 0c167869 (ine)
[CACHE MISS] ... 85950d51 (ufsc)
[CACHE MISS] ... d2184733 (eas) ← cache atinge 5
[CACHE MISS] ... 3424b0d8 (cse) ← chega 6º, dispara eviction
[CACHE] Capacidade atingida. Removendo entrada obsoleta: 6eb12957
[CACHE] Entrada 3424b0d8 invalidada com sucesso.
[CACHE MISS] ... 3424b0d8 (cse após remove → 404 do servidor)
```

...

## 4.3 Heterogeneidade — interoperação Python ↔ JavaScript

Python encurta, JS resolve o mesmo código, JS remove, Python tenta resolver de novo:

...

```
Codigo gerado pelo Python: 826da07c
JS resolveu: { url_original: 'https://heterogeneidade.test' }
JS resolveu de novo (HIT): { url_original: 'https://heterogeneidade.test', fonte: 'cache' }
JS removeu: { removido: true }
Python resolveu de volta: {'erro': 'Código não encontrado'}
```

...

O HIT do JS foi sobre uma entrada criada pelo Python: o cache do proxy é único e compartilhado entre clientes de qualquer linguagem.

## 4.4 Fila de Prioridades sob carga — variação de parâmetro

Subindo o proxy com `DEMO\_DELAY=0.5` (atraso de 500ms por requisição) e rodando o script `demos/demo\_prioridades.py`, que abre 15 conexões TCP em paralelo misturando os 3 tipos de operação em ordem cronológica embaralhada:

```
```bash
$ DEMO_DELAY=0.5 docker compose up -d proxy
$ python3 demos/demo_prioridades.py
```

...

...

=== ORDEM DE PROCESSAMENTO (resposta do worker) ===

```

T+ 517.8ms #06 encurta (prio=2) ← estava no worker quando a fila começou a encher
T+1026.9ms #08 resolve (prio=0)
T+1530.6ms #14 resolve (prio=0)
T+2036.6ms #11 resolve (prio=0)
T+2543.6ms #00 resolve (prio=0) ← TODOS os resolves saíram antes
T+3051.7ms #05 remove (prio=1)
T+3562.3ms #04 remove (prio=1)
T+4069.4ms #01 remove (prio=1) ← depois TODOS os removes
T+4579.4ms #07 encurta (prio=2)
...
T+7637.2ms #13 encurta (prio=2) ← encurtas por último

```

=== TEMPO MÉDIO POR PRIORIDADE ===

```

prio=0 (resolve): 1784.4ms (n=4)
prio=1 (remove ): 3561.1ms (n=3)
prio=2 (encurta): 5410.1ms (n=8)
'''

```

Depois que a fila começou a acumular, o worker processou estritamente em ordem de prioridade. Em média, um `resolve` foi atendido 3× mais rápido que um `encurta` apesar de terem chegado todos no mesmo intervalo de ~1ms.

A única "fora de ordem" foi a `#06` (encurta processada primeiro): chegou quando a fila ainda estava vazia e o worker a pegou imediatamente. Como a implementação não é preemptiva (não interrompemos uma requisição em andamento para atender uma de prioridade maior), isso é esperado e em carga estacionária o efeito de prioridade domina.

4.5 Variar o tamanho do cache

Demonstra mudança de parametrização sem rebuild:

```

'''bash
$ CACHE_CAPACITY=20 docker compose up -d proxy
$ docker compose exec proxy python -c "import proxy; print(proxy.CACHE_CAPACITY)"
20
'''

```

Com capacidade 20, o cenário da seção 4.2 não dispararia eviction (só 6 inserções). O mesmo vale para `REST\_SERVER\_URL`, `PROXY\_BIND\_PORT` e `DEMO\_DELAY`.

5. Conclusões e limitações

- Cache-Aside reduz tráfego ao servidor de forma mensurável. O contador `acessos` no servidor só incrementa em cache miss; as resoluções servidas pelo cache não chegam ao servidor.
- A invalidação no `remove` mantém coerência sem complicação — uma única chamada `cache.invalidate(codigo)` depois do DELETE bem-sucedido.
- A fila de prioridades introduz equidade entre tipos de requisição com custo baixo. No teste sob carga, `resolve` foi atendido em média 3× mais rápido que `encurta`.
- Heterogeneidade não é uma propriedade do servidor. O protocolo JSON/TCP é simples o bastante para qualquer linguagem com socket nativo implementar uma biblioteca em ~30 linhas.

- O cache do proxy é único e compartilhado entre todos os clientes, independente da linguagem, comprovado pelo teste em que uma entrada criada pelo cliente JS foi expulsa pelo LRU disparado por inserções do cliente Python.